Workflow and architecture patterns for working together 2: Architectures

Jacob Archambault

May 26, 2026

Outline

Outline

Outline

Outline

Challenges

- multiple teams working in the same repository

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
    - feedback delays
    - merge conflict regressions
    - sunk costs fallacy risks
- coupling of all services at the database layer
    - high storage costs

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
    - feedback delays
    - merge conflict regressions
    - sunk costs fallacy risks
- coupling of all services at the database layer
    - high storage costs
    - single point of failure risk

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
    - feedback delays
    - merge conflict regressions
    - sunk costs fallacy risks
- coupling of all services at the database layer
    - high storage costs
    - single point of failure risk
    - risk of cross-project interference

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
    - feedback delays
    - merge conflict regressions
    - sunk costs fallacy risks
- coupling of all services at the database layer
    - high storage costs
    - single point of failure risk
    - risk of cross-project interference

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver
- collaboration $\rightarrow$ business stream-aligned teams over activity oriented teams

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/"Microservices"

Outline

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\mathit{no\ conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(no\ conflicts) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(no\ conflicts) = \frac{1}{2^n}$ where $|C| = n$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(no\ conflicts) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(no\ conflicts) = \frac{1}{2^n}$ where $|C| = n$
  - $P(no\ conflicts) < 10\%$ where $n \geq 4$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(no\ conflicts) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(no\ conflicts) = \frac{1}{2^n}$ where $|C| = n$
  - $P(no\ conflicts) < 10\%$ where $n \geq 4$
  - $P(no\ conflicts) < 1\%$ where $n \geq 7$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\textit{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\textit{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
  - $P(\textit{no conflicts}) < 10\%$ where $n \geq 4$
  - $P(\textit{no conflicts}) < 1\%$ where $n \geq 7$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\textit{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\textit{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
    - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(no\ conflicts(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
  - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n-(2^{n_1}-1)-(2^{n_2}-1)-(2^{n_3}-1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n-2^{n_1}-2^{n_2}-2^{n_3}+3}$
  - $= \frac{1}{2^5-2^1-2^2-2^2+3}$
  - $= \frac{1}{32-2-4-4+3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
    - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
    - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
    - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$
    - $= \frac{1}{32 - 2 - 4 - 4 + 3}$
    - $= \frac{1}{32 - 7}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n-(2^{n_1}-1)-(2^{n_2}-1)-(2^{n_3}-1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n-2^{n_1}-2^{n_2}-2^{n_3}+3}$
  - $= \frac{1}{2^5-2^1-2^2-2^2+3}$
  - $= \frac{1}{32-2-4-4+3}$
  - $= \frac{1}{32-7}$
  - $= \frac{1}{25}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\mathit{no\ conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n-(2^{n_1}-1)-(2^{n_2}-1)-(2^{n_3}-1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n-2^{n_1}-2^{n_2}-2^{n_3}+3}$
  - $= \frac{1}{2^5-2^1-2^2-2^2+3}$
  - $= \frac{1}{32-2-4-4+3}$
  - $= \frac{1}{32-7}$
  - $= \frac{1}{25}$
  - $= 4\%$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(no\ conflicts(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
  - $= \frac{1}{2^n-(2^{n_1}-1)-(2^{n_2}-1)-(2^{n_3}-1)}$ where $n = |A|$ and $n_i = |A_i|$
  - $= \frac{1}{2^n-2^{n_1}-2^{n_2}-2^{n_3}+3}$
  - $= \frac{1}{2^5-2^1-2^2-2^2+3}$
  - $= \frac{1}{32-2-4-4+3}$
  - $= \frac{1}{32-7}$
  - $= \frac{1}{25}$
  - $= 4\%$

A naive probability theory for merge consistency: general definition for distinct branches diverging from a common ancestor

- Let A be the set of unmerged commits across k distinct branches $A_1 \cdots A_k$, where $A = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_k$, $n = |A|$, and $n_i = |A_i|$ for $1 \leq i \leq k$. Then

A naive probability theory for merge consistency: general definition for distinct branches diverging from a common ancestor

- Let A be the set of unmerged commits across k distinct branches $A_1 \cdots A_k$, where $A = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_k$, $n = |A|$, and $n_i = |A_i|$ for $1 \leq i \leq k$. Then

$$P(no\ conflicts) = \frac{1}{2^n - (\sum_{i=1}^{k} 2^{n_i}) + k}$$

- a simplified probability theory for incompatible changes - Cartesian products commits across n branches over time - Cartesian explosion, - intra vs. inter team communication

Outline

- compare total commits across team branch vs. others - compare non-merge commits to non-PR merge commits across team branch vs. others (this represents percentage of value-adding work to maintenance work) (git log –oneline –merges | grep -iv “merge pull request” | wc -l)
- compare time to merge across SpecFlow branches vs. others
- compare passing to failing pipeline runs across team branch vs.

others, for both simulation pipeline and unit test pipeline

Outline

Team composition

- all teams working in same git repository are in the same daily scrum

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Team composition: enabling teams

- Strategy will be rolled out first for claims, then reimbursements, then other transactions

Team composition: enabling teams

- Strategy will be rolled out first for claims, then reimbursements, then other transactions
- the 837 COB team will serve as an enabling team for the new branching strategy

Team composition: enabling teams

- Strategy will be rolled out first for claims, then reimbursements, then other transactions
- the 837 COB team will serve as an enabling team for the new branching strategy

Outline

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- every commit is pushed to origin after passing local testing

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- every commit is pushed to origin after passing local testing
- every git commit message begins with a Jira ticket number

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- every commit is pushed to origin after passing local testing
- every git commit message begins with a Jira ticket number

Outline

- Every change is tested by the test suite locally before committing

- tests are written against new code as that code is being written

Outline

- daily PRs - stacked PRs

Outline

- pipeline is definitive for deployment - daily stale branch job

Outline

Outline

Outline